```
5.times { print triangular_numbers.next.first, " " }
puts
```

produces:

```
1 3 6 10 15
```

Note that we're returning a two-element array to keep both values, so we need to call first on the result to get the actual number. We'll see a work around in a second.

Enumerator objects are also enumerable (that is to say, the methods available to enumerable objects are also available to them). That means we can use Enumerable's methods (such as first) on them:

```
triangular_numbers = Enumerator.produce([1, 2]) do |number, count|
  [number + count, count + 1]
end

p triangular_numbers.first(5).map { _1.first }
```

produces:

```
[1, 3, 6, 10, 15]
```

You have to be careful with enumerators that can generate infinite sequences. Some of the regular Enumerator methods such as count and select will happily try to read the whole enumeration before returning a result. If you want a version of select that works with infinite sequences, you need to use the lazy method of enumerable.

If you call Enumerator.lazy on any Ruby enumerator, you get back an instance of class Enumerator::Lazy. This enumerator acts just like the original, but it reimplements methods such as select and map so that they can work with infinite sequences. Putting it another way, none of the lazy versions of the methods actually consume any data from the collection until that data is requested, and then they only consume enough to satisfy that request. In other words, they are "lazy".

To work this magic, the lazy versions of the various methods do not return arrays of data. Instead, each returns a new enumerator that includes its own special processing—the select method returns an enumerator that knows how to apply the select logic to its input collection, the map enumerator knows how to handle the map logic, and so on. The result is that if you chain a bunch of lazy enumerator methods, what you end up with is a chain of enumerators—the last one in the chain takes values from the one before it, and so on.

Let's play with this a little. To start, let's create a class that generates a stream of integers…

```
class InfiniteStream
  def all
    Enumerator.produce(0) do |number|
      number += 1
    end.lazy
  end
end

p InfiniteStream.new.all.first(10)
```

produces:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```